

useState Hook - Complete Guide

Table of Contents

1. [Introduction](#)
 2. [Basic Concepts](#)
 3. [Deep Dive into useState](#)
 4. [Rules of useState](#)
 5. [Common Patterns](#)
 6. [Common Mistakes](#)
 7. [Interview Questions](#)
-

Introduction

useState is a React Hook that lets you add state to functional components. Before Hooks were introduced in React 16.8, state management was only possible in class components. useState revolutionized functional components by making them stateful.

Simple Definition

Think of useState as a way to remember values between function calls. Every time your component re-renders, the function runs again, but useState helps preserve specific values across these re-renders.

Basic Concepts

What is State?

State is data that changes over time in your component. When state changes, React re-renders the component to reflect the new data in the UI.

Basic Syntax

```
const [stateVariable, setStateFunction] = useState(initialValue);
```

- **stateVariable**: The current value of the state
- **setStateFunction**: Function to update the state
- **initialValue**: The starting value (can be any data type)

Simple Example

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

Deep Dive into useState

1. How useState Works Internally

The Fiber Architecture

React uses a data structure called **Fiber** to manage components. Each component has a corresponding Fiber node that stores:

- Component state
- Props
- Hooks information

Linked List Structure

When you use multiple useState hooks in a component, React maintains them in a **linked list**:

```
function Component() {
  const [name, setName] = useState("John"); // Hook 1 → Node 1
  const [age, setAge] = useState(25); // Hook 2 → Node 2
  const [city, setCity] = useState("New York"); // Hook 3 → Node 3
}
```

Internally, React creates a linked list:

```
Hook1 → Hook2 → Hook3 → null
```

Each node contains:

- Current state value
- Update queue (pending state updates)
- Pointer to next hook

Why Order Matters

React relies on the **call order** to match hooks between renders:

First Render:

```
useState("John")      // Position 0
useState(25)           // Position 1
useState("New York")  // Position 2
```

Second Render:

```
useState("John")      // Position 0 ✓
useState(25)           // Position 1 ✓
useState("New York")  // Position 2 ✓
```

If you break the order (e.g., conditional hooks), React gets confused:

✗ Wrong - Order Changes:

```
// First render
useState("John")      // Position 0
if (condition) {
  useState(25)         // Position 1
}
useState("New York")  // Position 2 or 1?

// Second render (condition = false)
useState("John")      // Position 0
// useState(25) is skipped!
useState("New York")  // Position 1 (expects 25, gets "New York")
```

2. State Updates Are Asynchronous

Why Asynchronous?

React batches state updates for performance. Instead of re-rendering after each `setState` call, React groups multiple updates together.

Example of Asynchronous Behavior

```
function Counter() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    console.log("Before:", count); // 0
    setCount(count + 1);
    console.log("After:", count); // Still 0! (not 1)
    // Component hasn't re-rendered yet
  };

  return <button onClick={handleClick}>Count: {count}</button>;
}
```

Batching Example

```
const handleClick = () => {
  setCount(count + 1); // count = 0, sets to 1
  setCount(count + 1); // count = 0, sets to 1 (not 2!)
  setCount(count + 1); // count = 0, sets to 1 (not 3!)
  // Final result: count = 1 (not 3!)
};
```

Why? All three calls use the initial `count` value (0) because the component hasn't re-rendered yet.

Solution: Functional Updates

```
const handleClick = () => {
  setCount(prevCount => prevCount + 1); // 0 + 1 = 1
  setCount(prevCount => prevCount + 1); // 1 + 1 = 2
  setCount(prevCount => prevCount + 1); // 2 + 1 = 3
  // Final result: count = 3 ✓
};
```

3. Initial Value Computation

Simple Initial Value

```
const [count, setCount] = useState(0);
```

Expensive Computation (✗ Inefficient)

```
// This runs on EVERY render!
const [data, setData] = useState(computeExpensiveValue());
```

Lazy Initial State (✓ Efficient)

```
// Function runs ONLY on first render
const [data, setData] = useState(() => computeExpensiveValue());
```

Example:

```
function HeavyComponent() {
  // ✗ Bad: Runs on every render
  const [items, setItems] = useState(
    Array(10000).fill(0).map((_, i) => ({ id: i, value: i * 2 })))
  );

  // ✓ Good: Runs only once
  const [items, setItems] = useState(() =>
    Array(10000).fill(0).map((_, i) => ({ id: i, value: i * 2 })))
  );
}
```

4. State Immutability

Why Immutability?

React compares old state with new state using reference equality. If you mutate the state directly, React won't detect the change.

✗ Mutating State Directly

```
const [user, setUser] = useState({ name: "John", age: 25 });

// Wrong - Mutates the original object
const updateAge = () => {
  user.age = 26;           // Mutates the object
  setUser(user);           // React won't detect change (same reference)
};
```

✓ Creating New Objects

```
const updateAge = () => {
  setUser({ ...user, age: 26 }); // Creates new object
};

// Or
const updateAge = () => {
  setUser(prevUser => ({ ...prevUser, age: 26 }));
};
```

Array Immutability

```
const [items, setItems] = useState([1, 2, 3]);

// ❌ Wrong - Mutates array
const addItem = () => {
  items.push(4);
  setItems(items);
};

// ✓ Correct - Creates new array
const addItem = () => {
  setItems([...items, 4]);
  // Or: setItems(prevItems => [...prevItems, 4]);
};

// Remove item
const removeItem = (index) => {
  setItems(items.filter((_, i) => i !== index));
};

// Update item
const updateItem = (index, newValue) => {
  setItems(items.map((item, i) => i === index ? newValue : item));
};
```

5. Closure and State State

The Closure Problem

```
function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      console.log(count); // Always logs 0!
      setCount(count + 1); // Always sets to 1!
    }, 1000);

    return () => clearInterval(interval);
  }, []); // Empty dependency - captures initial count (0)

  return <div>{count}</div>;
}
```

Solution 1: Use Functional Update

```
useEffect(() => {
  const interval = setInterval(() => {
    setCount(prevCount => prevCount + 1); // ✓ Works correctly
  }, 1000);

  return () => clearInterval(interval);
}, []);
```

Solution 2: Add Dependencies

```
useEffect(() => {
  const interval = setInterval(() => {
    setCount(count + 1); // Uses current count
  }, 1000);

  return () => clearInterval(interval);
}, [count]); // Re-creates interval when count changes
```

Rules of useState

1. Only Call Hooks at the Top Level

✓ **Correct:**

```
function Component() {
  const [name, setName] = useState("John");
  const [age, setAge] = useState(25);

  if (name === "John") {
    // Use state here is fine
    console.log(name);
  }

  return <div>{name}</div>;
}
```

✗ **Wrong:**

```
function Component() {
  if (condition) {
    const [name, setName] = useState("John"); // ✗ Conditional hook
  }

  for (let i = 0; i < 5; i++) {
    const [count, setCount] = useState(0); // ✗ Hook in loop
  }

  function nested() {
    const [data, setData] = useState(null); // ✗ Hook in nested function
  }

  return <div>Content</div>;
}
```

2. Only Call Hooks from React Functions

✓ **Correct:**

```
// React component
function MyComponent() {
  const [state, setState] = useState(0);
}

// Custom hook
function useCustomHook() {
  const [state, setState] = useState(0);
}
```

✗ Wrong:

```
// Regular JavaScript function
function regularFunction() {
  const [state, setState] = useState(0); // ✗
}

// Event handler (outside component)
const handleClick = () => {
  const [state, setState] = useState(0); // ✗
};
```

3. Don't Mutate State Directly

Always use the setter function to update state, never modify state variables directly.

4. State Updates May Be Batched

Multiple `setState` calls in the same event handler are batched together for performance.

Common Patterns

1. Object State

```
const [user, setUser] = useState({
  name: '',
  email: '',
  age: 0
});

// Update single property
const updateName = (newName) => {
  setUser(prevUser => ({
    ...prevUser,
    name: newName
  }));
};
```


2. Array State

```
const [todos, setTodos] = useState([]);

// Add item
const addTodo = (text) => {
  setTodos([...todos, { id: Date.now(), text, completed: false }]);
};

// Remove item
const removeTodo = (id) => {
  setTodos(todos.filter(todo => todo.id !== id));
};

// Toggle item
const toggleTodo = (id) => {
  setTodos(todos.map(todo =>
    todo.id === id ? { ...todo, completed: !todo.completed } : todo
  ));
};
```

3. Boolean Toggle

```
const [isOpen, setIsOpen] = useState(false);

const toggle = () => {
  setIsOpen(prev => !prev);
};
```

4. Counter with Limits

```
const [count, setCount] = useState(0);

const increment = () => {
  setCount(prev => Math.min(prev + 1, 100)); // Max 100
};

const decrement = () => {
  setCount(prev => Math.max(prev - 1, 0)); // Min 0
};
```

5. Derived State (Avoid When Possible)

```
// ✗ Redundant state
const [firstName, setFirstName] = useState('');
const [lastName, setLastName] = useState('');
const [fullName, setFullName] = useState(''); // Redundant!

// ✓ Calculate during render
const [firstName, setFirstName] = useState('');
const [lastName, setLastName] = useState('');
const fullName = `${firstName} ${lastName}`; // Derived value
```

Common Mistakes

1. Conditional Hooks

```
// ✗ Wrong
function Component({ showUsername }) {
  const [count, setCount] = useState(0);

  if (showUsername) {
    const [username, setUsername] = useState(''); // Order changes!
  }

  const [status, setStatus] = useState(false);
}

// ✓ Correct
function Component({ showUsername }) {
  const [count, setCount] = useState(0);
  const [username, setUsername] = useState('');
  const [status, setStatus] = useState(false);

  // Use conditional rendering instead
  return (
    <div>
      {showUsername && <input value={username} onChange={e => setUsername(e.target.value)} />}
    </div>
  );
}
```

2. Using State Value Immediately After Setting

```
// ❌ Wrong
const handleClick = () => {
  setCount(count + 1);
  console.log(count); // Still old value!
  sendToServer(count); // Sends old value!
};

// ✓ Correct
const handleClick = () => {
  const newCount = count + 1;
  setCount(newCount);
  console.log(newCount); // New value
  sendToServer(newCount); // Sends new value
};
```

3. Multiple Updates with Same Value

```
// ❌ Wrong - Only increases by 1
const handleClick = () => {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
};

// ✓ Correct - Increases by 3
const handleClick = () => {
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
};
```

4. Direct Mutation

```
const [items, setItems] = useState([1, 2, 3]);

// ❌ Wrong
const addItem = () => {
  items.push(4);
  setItems(items);
};

// ✓ Correct
const addItem = () => {
  setItems([...items, 4]);
};
```

Interview Questions

Question 1: What is useState and why do we need it?

Answer: useState is a React Hook that allows functional components to have state. Before Hooks, state management was only possible in class components. useState returns an array with two elements:

1. The current state value
2. A function to update that state

When state changes, React re-renders the component to reflect the new state in the UI.

```
const [count, setCount] = useState(0);
```

We need useState because:

- Functional components don't have `this.state` like class components
- We need to preserve values across re-renders
- It triggers re-renders when state changes
- It's simpler and more intuitive than class-based state management

Question 2: Why can't we call useState inside conditions or loops?

Answer: React relies on the order in which Hooks are called to maintain state consistency across renders. Internally, React uses a linked list to store hooks, and it matches hooks based on their call order, not their names.

If you call useState conditionally:

```
if (condition) {  
  const [name, setName] = useState(''); // Position might change!  
}
```

The order of hooks can change between renders, causing React to associate state with the wrong variable. This breaks React's internal state management.

Why it matters:

- First render: condition = true → useState at position 1
- Second render: condition = false → useState skipped → order mismatch
- React can't reliably match state to the correct variable

Rule: Always call hooks at the top level of your component, in the same order every render.

Question 3: What's the difference between these two approaches?

```
// Approach 1
setCount(count + 1);

// Approach 2
setCount(prevCount => prevCount + 1);
```

Answer: The difference is in how they handle state updates:

Approach 1 - Direct Value:

- Uses the current value of `count` from the closure
- Doesn't work correctly with multiple updates in the same render
- Can lead to stale state issues in async operations

Approach 2 - Functional Update:

- Uses the most recent state value React has
- Works correctly with multiple sequential updates
- Recommended for updates based on previous state
- Prevents closure/stale state problems

Example showing the difference:

```
// Direct value - only increments by 1
const handleClick = () => {
  setCount(count + 1); // count = 0, sets to 1
  setCount(count + 1); // count = 0, sets to 1
  setCount(count + 1); // count = 0, sets to 1
  // Result: count = 1
};

// Functional update - increments by 3
const handleClick = () => {
  setCount(prev => prev + 1); // 0 + 1 = 1
  setCount(prev => prev + 1); // 1 + 1 = 2
  setCount(prev => prev + 1); // 2 + 1 = 3
  // Result: count = 3
};
```

Question 4: Are state updates synchronous or asynchronous? Explain.

Answer: State updates are **asynchronous** and **batched** for performance optimization.

What this means:

1. When you call `setState`, React doesn't update the state immediately
2. React schedules an update and batches multiple `setState` calls together

3. The component re-renders once with all batched updates applied
4. You can't access the new state value immediately after calling `setState`

Example:

```
const handleClick = () => {
  console.log("Before:", count); // 0
  setCount(count + 1);
  console.log("After:", count); // Still 0, not 1!
  // The state hasn't updated yet
};
```

In React 18+:

- Automatic batching happens in all event handlers, timeouts, promises, and native events
- In React 17 and earlier, batching only occurred in React event handlers

To access new state after update:

```
// Option 1: Use the new value directly
const handleClick = () => {
  const newCount = count + 1;
  setCount(newCount);
  console.log(newCount); // New value
};

// Option 2: Use useEffect
useEffect(() => {
  console.log("Count updated:", count);
}, [count]);
```

Question 5: What happens if you mutate state directly instead of using `setState`?

Answer: Directly mutating state is a critical mistake that breaks React's rendering mechanism.

What happens:

```
// ❌ Wrong
const [user, setUser] = useState({ name: 'John', age: 25 });

const updateAge = () => {
  user.age = 26; // Mutates the original object
  setUser(user); // React won't detect the change!
};
```

Why it fails:

1. React uses shallow comparison (reference equality) to detect changes
2. When you mutate the object, the reference stays the same

3. React thinks nothing changed: `oldUser === newUser` (same reference)
4. No re-render is triggered
5. UI doesn't update even though the data changed

Correct approach:

```
// ✓ Correct - Create new object
const updateAge = () => {
  setUser({ ...user, age: 26 }); // New object, new reference
};
```

Same applies to arrays:

```
// ✗ Wrong
items.push(newItem);
setItems(items);

// ✓ Correct
setItems([...items, newItem]);
```

Question 6: How does React keep track of multiple useState hooks in a component?

Answer: React uses a **linked list data structure** to maintain hooks for each component instance.

How it works:

1. First Render:

- React creates a Fiber node for the component
- Each useState call adds a node to the linked list
- Nodes are stored in call order

```
function Component() {
  const [name, setName] = useState('John'); // Node 0
  const [age, setAge] = useState(25);       // Node 1
  const [city, setCity] = useState('NYC');  // Node 2
}
```

Internal structure:

```
Fiber {
  hooks: Node0 → Node1 → Node2 → null
}

Node0: { state: 'John', next: Node1 }
Node1: { state: 25, next: Node2 }
Node2: { state: 'NYC', next: null }
```

2. Subsequent Renders:

- React traverses the linked list in order
- Matches hooks by position (not by variable name)
- Returns the stored state value

This is why:

- Hook order must remain consistent
- Conditional hooks break the traversal
- React doesn't need variable names to track state

The cursor mechanism: React maintains a cursor that points to the current hook:

- Render starts → cursor at position 0
- First `useState` → returns `Node0`, cursor moves to `Node1`
- Second `useState` → returns `Node1`, cursor moves to `Node2`
- Render ends → cursor resets

Question 7: What is lazy initialization in `useState` and when should you use it?

Answer: Lazy initialization is a performance optimization where the initial state is computed using a function that runs **only once** on the first render.

Syntax:

```
// Regular initialization - runs on EVERY render
const [state, setState] = useState(expensiveComputation());

// Lazy initialization - runs ONLY on first render
const [state, setState] = useState(() => expensiveComputation());
```

When to use:

1. Expensive computations:


```
const [data, setData] = useState(() => {
  // This only runs once
  return Array(10000).fill(0).map((_, i) => ({
    id: i,
    value: Math.random()
  }));
});
```

2. Reading from localStorage:

```
const [theme, setTheme] = useState(() => {
  // Only reads from localStorage once
  return localStorage.getItem('theme') || 'light';
});
```

3. Complex object creation:

```
const [config, setConfig] = useState(() => {
  // Heavy processing only on mount
  return processConfiguration(defaultConfig);
});
```

Performance impact:

```
// ✗ Without lazy init - runs on every render
function Component() {
  const [items, setItems] = useState(heavyComputation()); // Runs every render!
  // If component re-renders 10 times, heavyComputation() runs 10 times
}

// ✓ With lazy init - runs once
function Component() {
  const [items, setItems] = useState(() => heavyComputation()); // Runs only once!
  // Component can re-render 100 times, heavyComputation() still runs only once
}
```

Question 8: Explain the concept of "stale closure" in relation to useState.

Answer: A stale closure occurs when a function captures an old value of state and continues to use that outdated value, even after the state has changed.

Example of stale closure:

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      console.log(count); // Always logs 0!
      setCount(count + 1); // Always sets to 1!
    }, 1000);

    return () => clearInterval(interval);
  }, []); // Empty deps - closure captures count = 0 forever

  return <div>{count}</div>;
}
```

Why it happens:

1. useEffect runs once (empty dependency array)
2. The callback captures `count = 0` in its closure
3. Even though count changes in the UI, the interval callback still references the original `count = 0`
4. It's "stale" because it doesn't see the updated count value

Solutions:

Solution 1: Functional updates

```
useEffect(() => {
  const interval = setInterval(() => {
    setCount(prevCount => prevCount + 1); // ✓ Always uses latest state
  }, 1000);

  return () => clearInterval(interval);
}, []); // Can keep empty deps
```

Solution 2: Add to dependencies

```
useEffect(() => {
  const interval = setInterval(() => {
    setCount(count + 1); // Uses current count
  }, 1000);

  return () => clearInterval(interval);
}, [count]); // Re-creates interval with fresh closure each time count changes
```

Solution 3: useRef for the latest value

```
function Counter() {
  const [count, setCount] = useState(0);
  const countRef = useRef(count);

  useEffect(() => {
    countRef.current = count; // Always update ref with latest count
  }, [count]);

  useEffect(() => {
    const interval = setInterval(() => {
      console.log(countRef.current); // ✓ Always logs latest count
      setCount(countRef.current + 1);
    }, 1000);

    return () => clearInterval(interval);
  }, []);
}
```

Question 9: Can you update state based on multiple previous state values? How?

Answer: Yes, you can update state based on multiple previous state values using functional updates or by combining states.

Approach 1: Multiple functional updates

```
function Component() {
  const [count, setCount] = useState(0);
  const [multiplier, setMultiplier] = useState(2);

  const updateCount = () => {
    setCount(prevCount => prevCount + 1);
    setMultiplier(prevMultiplier => prevMultiplier + 1);
  };
}
```

Approach 2: Single state object

```
function Component() {
  const [state, setState] = useState({
    count: 0,
    multiplier: 2
  });

  const updateBoth = () => {
    setState(prevState => ({
      count: prevState.count + 1,
      multiplier: prevState.multiplier + 1
    }));
  };

  const calculateTotal = () => {
    setState(prevState => ({
      ...prevState,
      total: prevState.count * prevState.multiplier
    }));
  };
}
```

Approach 3: Derived state calculation

```
function Component() {
  const [count, setCount] = useState(0);
  const [multiplier, setMultiplier] = useState(2);

  const incrementBoth = () => {
    setCount(prev => prev + 1);
    setMultiplier(prev => {
      // Can access local variables in the same render
      const newCount = count + 1; // This is still the old value
      return prev + 1;
    });
  };

  // Better: calculate derived value during render
  const total = count * multiplier;
}
```

Approach 4: useReducer for complex logic

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'INCREMENT_BOTH':
      return {
        count: state.count + 1,
        multiplier: state.multiplier + 1,
        total: (state.count + 1) * (state.multiplier + 1)
      };
    default:
      return state;
  }
};

function Component() {
  const [state, dispatch] = useReducer(reducer, {
    count: 0,
    multiplier: 2,
    total: 0
  });

  const incrementBoth = () => {
    dispatch({ type: 'INCREMENT_BOTH' });
  };
}
```

Key point: When you need complex state logic involving multiple related values, consider using `useReducer` instead of multiple `useState` calls.

Question 10: What's the difference between `useState` and `useMemo` for derived values?

Answer: `useState` and `useMemo` serve different purposes when dealing with values in components.

`useState` - For State (Values that trigger re-renders):

```
const [count, setCount] = useState(0);
```

- Stores mutable state
- Triggers re-render when updated
- Value persists across re-renders
- Use when you need to update the value

`useMemo` - For Expensive Computations (Performance optimization):

```
const expensiveValue = useMemo(() => {
  return computeExpensiveValue(a, b);
}, [a, b]);
```

- Caches computed values
- Doesn't trigger re-renders

- Recomputes only when dependencies change
- Use for performance optimization

When NOT to use useState for derived values:

```
// ✗ Bad - Redundant state
function Component({ items }) {
  const [itemCount, setItemCount] = useState(0);

  useEffect(() => {
    setItemCount(items.length); // Unnecessary!
  }, [items]);

  return <div>Count: {itemCount}</div>;
}

// ✓ Good - Just calculate it
function Component({ items }) {
  const itemCount = items.length; // Simple calculation
  return <div>Count: {itemCount}</div>;
}
```

When to use useMemo:

```
// ✓ Good - Expensive computation
function Component({ items }) {
  const sortedItems = useMemo(() => {
    console.log('Sorting...');
    return [...items].sort((a, b) => a.value - b.value);
  }, [items]); // Only re-sort when items change

  return <div>{sortedItems.map(item => ...)}</div>;
}
```

Decision tree:

1. **Can the value be calculated from props/state?** → Just calculate it (no hook needed)
2. **Is the calculation expensive?** → Use `useMemo`
3. **Do you need to update the value independently?** → Use `useState`
4. **Does the value trigger side effects?** → Use `useEffect` with the calculated/memoized value

Examples:

```
function ShoppingCart({ items, taxRate }) {
  // ✓ Simple derivation - no hook needed
  const subtotal = items.reduce((sum, item) => sum + item.price, 0);

  // ✓ Simple calculation - no hook needed
  const tax = subtotal * taxRate;
  const total = subtotal + tax;

  // ✓ useMemo for expensive operation
  const recommendations = useMemo(() => {
    return getRecommendations(items); // Expensive API/computation
  }, [items]);

  // ✓ useState for user interactions
  const [discountCode, setDiscountCode] = useState('');

  return (
    <div>
      <p>Subtotal: ${subtotal}</p>
      <p>Tax: ${tax}</p>
      <p>Total: ${total}</p>
      <input
        value={discountCode}
        onChange={e => setDiscountCode(e.target.value)}
      />
    </div>
  );
}
```

Summary

Key Takeaways:

1. **useState** adds state to functional components
2. Always call hooks at the **top level** (no conditions/loops)
3. State updates are **asynchronous** and **batched**
4. Use **functional updates** for updates based on previous state
5. Never **mutate** state directly - always create new objects/arrays
6. React tracks hooks using a **linked list** based on **call order**
7. Use **lazy initialization** for expensive initial values
8. Watch out for **stale closures** in async operations
9. Keep state **minimal** - derive values when possible
10. Consider **useReducer** for complex state logic

Best Practices:

- ✓ Keep state minimal and flat
- ✓ Use functional updates for state that depends on previous state
- ✓ Create new objects/arrays instead of mutating
- ✓ Use lazy initialization for expensive computations

- ✓ Lift state up when multiple components need it
 - ✓ Consider useReducer for complex state logic
 - ✗ Don't use state for values that can be calculated
 - ✗ Don't call hooks conditionally
 - ✗ Don't mutate state directly
 - ✗ Don't assume state updates are immediate
-